

Virtual Diablo 630 Printer

June 19, 2026

Table of Contents

Overview.....	1
Common properties.....	1
Diablo630telnet.jar properties.....	3
Diablo630tty.jar properties.....	4
Special ESC Commands.....	4
GUI Menus.....	4
Notes On Fonts.....	4
Proportional Fonts.....	5
Builtin Fonts.....	5

Overview

This device accepts text and ESC sequences for a Diablo 630 Printer, or one of it's clones, and produces PostScript output to a file. Through the use of "job end" actions, that output may be printed or otherwise handled.

Printer output is accumulated until an End Job code (0xff) is received, at which point the postscript output is saved and any Job End actions are taken (like submitting to a printer). Alternatively, a Cancel Job code (0xfe) may be sent which will discard current output without taking any actions. Without one of these codes, printer output (postscript) will continue to accumulate in the temporary file.

Common properties

Property values that allow substitutions recognize the following:

- %f** Job file. Typically the **diablo630_file** parameter (after substitutions).
- %b** Job file without suffix, if any.
- %x** Use built-in temporary file creation. All other tags are ignored. There must be at least 3 characters before the **%x**.
- %j** Job ID. Resets back to "1" whenever JAR is started, however the presence of this sub triggers a scan of existing files to determine the last used job ID.
- %s** Timestamp as in default.
- %t** Time in the form HHMMSS.S.
- %d** Date in the form YYYYMMDD.

conf=*file*

(commandline only) The file to use for properties/configuration. Default is **diablo630.rc** in the current directory, or else **~/.diablo630rc**.

diablo630_cpi=*cpi*

(or "cpi=") The default CPI setting, either 10, 12, or 15.

diablo630_lpi=*lpi*

(or "lpi=") The default (initial) LPI setting, either 6 or 8.

diablo630_scale=*factor*

(or "scale=") Scaling factor to use on the page. Default is "1.0" or no scaling.

diablo630_left=*inches*

(or "left=") Define first printing position to be *inches* inside of the left edge of the paper. This is

analogous to the horizontal registration of the paper in the printer.

diablo630_top=*inches*

(or "top=") Define first printing position to be *inches* inside of the top edge of the paper. This is analogous to the vertical registration of "top of form" on the printer.

diablo630_font=*params...*

(or "font=", comma-separated) The default (initial) Font to use. Syntax is

Name Style Size

(or "Name,Style,Size") Where "Name" is a valid string for the host computer's installed fonts. Also accepts Java generic fonts such as "Monospaced". "Style" is things like "PLAIN". "Size" is the point size, typically things like "12" for 10 cpi printing.

diablo630_alt_color=*RGB*

(or "alt_color=") The alternate color to use instead of RED for two-color printing. A 6-digit hexadecimal number representing the RGB color. Default is ff0000 or RED.

diablo630_paper=*params...*

(or "paper=", comma-separated) The default (initial) Paper to use. Syntax is

Param Param...

(or "param,param...") Where params are some meaningful combination of:

LETTER

U.S. Letter sized paper, 8.5x11

LEGAL

U.S. Legal sized paper, 8.5x14

A4

ISO A4 sized paper, 210x297mm

FORMS

Legacy large forms sized paper, 11x14, typically used LANDSCAPE

PORTRAIT

Portrait orientation

LANDSCAPE

Landscape orientation

Other media sizes may be selected from the "Paper" option in the "Settings" menu.

diablo630_file=*file-pattern*

(or "file=") The initial file to use for output. Especially handy when used as a parameter for multiple printers on a server, to ensure each has a unique and separate output file. Allows substitutions as defined above, although use of %f is not recommended. This feature largely obsoletes **jobend=SAVE**. Default is "out.ps". To preserve output of all jobs, use something like "job%j.ps".

diablo630_nogui

(or "nogui") Do not create/display the printer control console. Useful for CP/Net server applications, especially if multiple printers are being created.

diablo630_jobend=*params...*

(or "jobend=") The action to take when a job ends (e.g. ENDLIST.COM is run). The can only be set in a configuration (properties) file. Value syntax is one or more of:

DISCARD

Throw away output and start next job with empty file. Not terribly useful. The file will not be

overwritten until the next job ends.

SAVE [*file-pattern*]

This is largely obsolete due to **diablo630_file** allowing substitutions. Rename file to a "unique" name. Default filename pattern is "jobYYYYMMDDHHMMSS.S.ps", where the upper-case letters represent the digits of the current date and time (down to milliseconds). The *file-pattern* may contain substitution tags as defined above.

If more actions follow, *pattern* must be "-" to choose default.

QUEUE *command...*

Execute *command*, presumably to queue the output to a printer. The command is scanned for the same tags as for **SAVE** except that **%f** (and **%b**) may be the file name specified by **SAVE** (if present). This command should not produce any stdout or stderr output, and should return promptly (i.e. be well-behaved). Typical command for a Linux system with CUPS, and having a default printer set, might be "lpr %f". The printer queue must be capable of (possibly using special options) processing PostScript. No special shell syntax is allowed, such as redirection, variable substitution, or quoting. The command itself, though, may be a shell script (for example, a wrapper around some more complex command). As an alternate example to submitting job to a printer, this command would convert the postscript to PDF:

```
ps2pdf %f %b.pdf
```

If more actions follow, *command...* must end with "--".

If both **SAVE** and **QUEUE** are specified, the job will be renamed first and then the queue command run on the saved file name.

If conflicting actions are assigned (e.g. **DISCARD** and **SAVE**), results are unspecified.

diablo630_esc_h=unicode

diablo630_esc_i=unicode

diablo630_esc_j=unicode

diablo630_esc_k=unicode

diablo630_esc_y=unicode

diablo630_esc_z=unicode

(or "esc_x=") The unicode value for the character glyph to use for the Juki ESC H (I, J, K, Y, Z) sequence. Defaults are values for the standard Juki definitions: 0x00A7, 0x00A3, 0x00A8, 0x00E7, 0x00A2, 0x00AC (§ £ ¨ ¤ ¤ ¬) respectively.

diablo630_tee=file

(or "tee=") Collect all input to *file*. For debugging purposes. Raw characters and ESC sequences from the host are saved to the file.

diablo630_auto_lf

(or "auto_lf") A LF is performed on every CR received. Default is off.

diablo630_col132_nl

(or "col132_nl") If attempting to print beyond column 132, force a CR/LF. Implemented as a CR/LF when printing beyond the right edge of the paper. Default is off.

Diablo630telnet.jar properties

diablo630_host=host

(or "host=") Use the specified host (or IP address) for the connection.

diablo630_port=port

(or "port=") Use the specified TCP/IP port number for the connection.

Diablo630tty.jar properties

diablo630_tty=*device*

(or "tty=") Use the specified serial port for the connection.

diablo630_baud=*speed*

(or "baud=") Use the specified serial port speed for the connection.

Special ESC Commands

The follow special ESC commands are recognized (not part of the Diablo 630 or Juki 6100 specification).

ESC F

Begin upload of new properties. All following text becomes a new set of properties that will be applied to the default settings. Only print related properties may be changed (none of **diablo630_file**, **diablo630_nogui**, **diablo630_jobend**, **diablo630_tee**, nor those for telnet or tty variants).

ESC G

Stop uploading new properties and apply the current set. Printer settings are reset to the defaults and this set of properties are applied. Any properties in a config file or on the commandline are not applied.

ESC V

Restore properties to those applied at startup. This effectively restores the settings used to start the program (defaults plus any properties in a config file or on the commandline)

GUI Menus

File

New Output File – Set new output file name, overriding **diablo630_file**.

Discard – Discard current output. Same as if the CANCEL_JOB character (0xfe) was received.

Save Prop Table – (if font is proportional) Dump the proportional spacing table of the current font for characters 0x20 (space) through 0x7f (DEL) to the selected file (overwritten). Units are 1/120 inch.

Settings

Paper – Update Paper settings Media, Orientation, Scaling, Left/Top offsets.

Font – Change font settings Name, Size, Bold/Italics, cpi and lpi.

Controls

Form Feed – Same as if a FF was received on print stream.

End Job – Same as if the END_JOB character (0xff) was received.

Notes On Fonts

JAVA defines the logical fonts **Serif**, **SansSerif**, **Monospaced**, **Dialog**, and **DialogInput**. These get mapped to physical fonts that are available by the JRE/JVM. In addition, JAVA makes available fonts that are installed on your local operating system. The efficiency of the generated postscript depends on the fonts chose, with greatest efficiency for documents using the JAVA logical fonts. If the chosen font is not one available to the JAVA print facilities, then each character will be drawn explicitly each time, causing a much larger postscript file. The best way to ensure a builtin font is used is to use a JAVA

logical font name. Or, creating a simple postscript output and examining that with a text editor will reveal the list of available fonts in the `/FL [ ... ] D` expression block.

Font sizes are defined by the number of “points” (1/72 inch) measuring the height of the overall font, which differs from legacy printing which was concerned with CPI (characters/inch) and LPI (lines/pinch). A rough gauge is that a 12 point mono-spaced font is generally close to 10 cpi, however that depends largely on the characteristics of the font. An attempt is made to adjust the character width of mono-spaced fonts to match the selected CPI.

Proportional Fonts

Mono-spaced fonts most-closely match the legacy printers (and thus legacy printing models), where CPI and LPI have well defined meanings. The concept of proportional spacing was introduced to most word processing programs, but other modes of printing (such as using PIP to send plain text to LST:) does not handle that well.

Most modern proportional fonts define their character widths in fractional terms (floating point). All known legacy word processors have proportional spacing tables of integers. Internally, this printer keeps the table as floating point numbers so there may be discrepancies when dealing with word processors.

Some special commandline args allow generation of character width tables for proportional fonts, to be used to customize word processing programs. Data is only generated for characters 0x20 (space) through 0x7f (DEL). Specifying one of these causes the printer program to immediately exit after printing the table to stdout.

genprop – generate a simple numerical list of integer character widths in units of 1/120 inch.

genmw – generate the proportional spacing table for Magic Wand, in Hex ready for overlaying PRINT.COM using DDT. Generates the entire table at 0x500, although the first 32 characters (control codes) are always zeroes. Data is in units of 1/120 inch.

genws[=org] – generate the proportional spacing table for WordStar, in HEX ready for overlaying WS.COM using DDT. The default *org* is 0x7ba which aligns with WS.COM version 3.30. A different *org* may be specified for other versions. This table is labeled “experimental/unsupported” in documentation for WordStar 3.0. Data is in units of 1/60 inch minus 3 (internal space value), stored in the lower nibble of each byte.

For example, to generate the Magic Wand table for the logical font “Serif” use this command:

```
java -jar Diablo630.jar nogui font=Serif,plain,12 genmw >mwserif.hex
```

Builtin Fonts

Builtin fonts are the most efficient for size of postscript output, where the font can be referenced by index number and characters simply “drawn” by ASCII character code.

If the font is not builtin, the postscript will contain commands to draw each character explicitly, using line/arc/fill drawing expressions. In addition, it appears that the postscript generator does not simply define the font and later reference each character glyph, but rather just draws each character separately. This produces a much larger postscript file.